Shaun Joe Roy

Infrastructure / SRE Intern · Assignment Submission

Zeotap · May 2026

Mission-Critical Incident Management System

A high-throughput, distributed incident lifecycle platform built on Fastify, BullMQ, PostgreSQL / TimescaleDB, MongoDB, Redis, and React — handling 10,000 signals/sec with full RCA enforcement.

GitHub Repository

github.com/joery0x3b800001/Mission-Critical-IMS

Submitted to karan.sinha@zeotap.com · Deadline 6 May 2026

1. PROJECT OVERVIEW

This submission presents a **Mission-Critical Incident Management System (IMS)** designed to monitor a complex distributed stack — APIs, MCP Hosts, Distributed Caches, Async Queues, RDBMS, and NoSQL stores — and manage failure mediation workflows from first signal through to a closed incident with a mandatory Root Cause Analysis.

The system sustains **10,000 signals/sec** (verified), implements intelligent debouncing, persists data across three purpose-specific stores, and exposes a real-time React dashboard with live WebSocket updates.

2. SYSTEM ARCHITECTURE

```
Signal Producers → Fastify API → BullMQ (Redis) → Worker Pool → PostgreSQL + MongoDB + TimescaleDB → React Dashboard
```

The architecture centres on a **decoupled ingestion pipeline**. The HTTP layer responds with **202 Accepted** immediately — it never blocks on database writes. BullMQ acts as the durable buffer between the fast ingestion path and slower persistence.

Layer	Technology	Role
API Server	Fastify + TypeScript	Rate limiting, Zod validation, 202 immediate ACK
Queue / Buffer	BullMQ + Redis	Backpressure absorption, retryable jobs, priority queuing
Worker Pool	BullMQ Workers	Concurrent signal processing, debounce logic, alert routing
Source of Truth	PostgreSQL + TimescaleDB	Work Items, RCA records, timeseries signal metrics (ACID)
Audit Log	MongoDB	Raw signal payloads, queryable by workItemId / componentId
Hot Cache	Redis	Debounce windows (atomic INCR + TTL), dashboard state
Frontend	React + Vite + Tailwind	Live feed, incident detail, RCA form with live validation
Real-time	WebSocket	Push-based dashboard updates — no polling required

3. DESIGN PATTERNS (LLD)

3.1 Strategy Pattern — Alerting

The **AlertStrategy** interface decouples alert logic from the signal processor. The correct strategy is resolved at runtime via a factory keyed on **componentType**, allowing alerting behaviour to be swapped without modifying processing code.

Strategy	Component Types	Priority	Action
P0AlertStrategy	RDBMS, MCP_HOST	**P0 — Critical**	Page on-call engineer
P1AlertStrategy	API, QUEUE	**P1 — High**	Notify Slack #incidents

Strategy	Component Types	Priority	Action
P2AlertStrategy	CACHE, NOSQL	**P2 — Medium**	Auto-create Jira ticket

3.2 State Pattern — Work Item Lifecycle

Each state class encapsulates valid transitions and throws on invalid attempts — no scattered if/switch logic. **RESOLVED** → **CLOSED** is the only transition requiring a complete RCA record; missing RCA returns HTTP 422. MTTR is calculated and persisted atomically on close.

```
OPEN -> INVESTIGATING -> RESOLVED -> CLOSED ^ Requires complete RCA record MTTR = incident_end - work_item.start_time
```

4. BACKPRESSURE & MEMORY MANAGEMENT

Five layers work together to sustain 10,000 signals/sec without memory exhaustion, even when the persistence layer is slow:

– **Rate Limiter** — Redis-backed global cap at 10,000 req / 10 s. Returns 429 before signals reach BullMQ.

– **Batch Endpoint** — POST /signals/batch accepts 100 signals per call. Reduces TCP overhead 100× via addBulk().

– **BullMQ Queue** — Redis-backed durable buffer. Jobs survive server restarts; slow Postgres never causes timeouts.

– **Worker Concurrency** — Configurable via WORKER_CONCURRENCY (defaults to CPU cores × 2).

– **Exponential Backoff** — Failed jobs retry 3× with delays of 500 ms, 1 s, 2 s. Only transient errors are retried.

5. DEBOUNCE LOGIC

A Redis atomic INCR with a 10-second TTL ensures that 100+ signals arriving for the same componentId within 10 seconds create exactly **one Work Item**. All signals are linked to that Work Item in MongoDB.

```
INCR ims:debounce:{componentId} (expires 10 s) count == 1 -> CREATE Work Item in Postgres (transactional) count > 1 -> INCREMENT signal_count only All signals -> INSERT into MongoDB raw_signals (always)
```

6. DATA HANDLING — THREE-STORE SEPARATION

Store	Data	Rationale
PostgreSQL + TimescaleDB	work_items, rca_records, signal_metrics hypertable	ACID transactions for state transitions. TimescaleDB for compressed timeseries aggregations.
MongoDB	raw_signals — every signal payload	Flexible schema, high write throughput, queryable by workItemId / componentId / timestamp.

Store	Data	Rationale
Redis	Debounce windows, rate-limit counters, dashboard cache	O(1) atomic operations. AOF persistence enabled — survives server restart.

7. SECURITY HARDENING

– **Zod input validation** — componentId regex, errorCode uppercase, message length cap (1000 chars), latencyMs range 0–60,000.

– **@fastify/helmet** — HSTS, X-Content-Type-Options, X-Frame-Options, and CSP on every response.

– **Body size limit** — 1 MB bodyLimit prevents memory exhaustion from oversized payloads.

– **Parameterised queries** — All Postgres queries use $1/$2 placeholders; zero string concatenation.

– **Statement timeout** — Postgres pool configured with statement_timeout: 10,000 ms.

– **Optional API Key auth** — X-API-Key header validation enabled via ENABLE_API_KEY_AUTH=true.

– **CORS lockdown** — Production restricts origin to http://localhost:3000.

– **Transient-only retries** — Retry checks error codes (40001, 40P01, 08006); constraint violations are not retried.

8. TESTING STRATEGY

Test File	Cases	Coverage
workItemState.test.ts	18+	All state transitions, RCA guard, MTTR calculation, transaction rollback
alertStrategy.test.ts	20+	P0/P1/P2 assignment per component type, context switching, title formatting
signals.test.ts	25+	Schema validation, priority queuing, edge cases, concurrent submissions
integration.test.ts	10+	Signal to Work Item creation, debounce behaviour, error recovery
stress.test.ts	8	Burst load (500 signals), sustained 500/sec, mixed types, 15-second stability

```
cd backend && npm test # run all 80+ tests npm run test:coverage # with coverage report
```

9. SETUP & RUNNING

9.1 Prerequisites

– Docker Desktop (includes Docker Compose v2)

– Node.js 22+ via nvm — nvm install --lts

9.2 Start the full stack

```
git clone https://github.com/joery0x3b800001/Mission-Critical-IMS cd Mission-Critical-IMS
docker compose up --build
```

Service	URL
Dashboard	http://localhost:3000
Backend API	http://localhost:3001
Health Check	http://localhost:3001/health
WebSocket	ws://localhost:3001/ws

9.3 Simulation scripts

```
cd scripts && npm install # Cascading outage scenario (640 signals, 5 components) npx ts-node simulate-outage.ts # 10,000 signals/sec burst test (50,000 total) npx ts-node simulate-outage.ts --burst # Reset all data between runs ./reset-data.sh
```

10. API REFERENCE

Method	Endpoint	Description
POST	/signals	Ingest one signal — 202 Accepted immediately
POST	/signals/batch	Ingest up to 100 signals in one HTTP call (bulk high-throughput path)
GET	/incidents	List all incidents sorted by priority (P0 first) then start time
GET	/incidents/:id	Incident detail + raw signals from MongoDB
PATCH	/incidents/:id/status	State transition — enforced by State Pattern (422 on invalid)
POST	/incidents/:id/rca	Submit or update RCA record. Required before CLOSED transition.
GET	/health	Health status, queue depth, per-service ping, uptime

11. EVALUATION — SELF ASSESSMENT

Category	Wt.	Implementation	Status
Concurrency & Scaling	10 %	BullMQ worker pool (CPU×2), atomic Redis debounce, 10k/sec burst verified	Done
Data Handling	20 %	3 stores: PostgreSQL (source of truth), MongoDB (audit), Redis (cache). TimescaleDB hypertable.	Done
LLD	20 %	Strategy Pattern (alerting), State Pattern (lifecycle), withTransaction, Zod validation	Done
UI / UX & Integration	20 %	React dashboard: live feed (P0 first), incident detail, RCA form, WebSocket, field grayout	Done
Resilience & Testing	10 %	80+ tests (unit + integration + stress), exponential backoff, /health, graceful shutdown	Done

Category	Wt.	Implementation	Status
Documentation	10 %	README with architecture diagram, backpressure, API reference, troubleshooting. SECURITY.md, PERFORMANCE.md checked in.	Done
Tech Stack Choices	10 %	Fastify (fastest Node HTTP), BullMQ, TimescaleDB, MongoDB, React + Vite	Done
Bonus	+	Helmet headers, batch endpoint, Artillery load test config, simulate-outage --burst, live form grayout	Bonus

12. DASHBOARD — SCREENSHOTS

The React frontend connects via WebSocket and updates in real time. Below are three annotated views captured during a live burst-test run (50,000 signals, 5 component types).

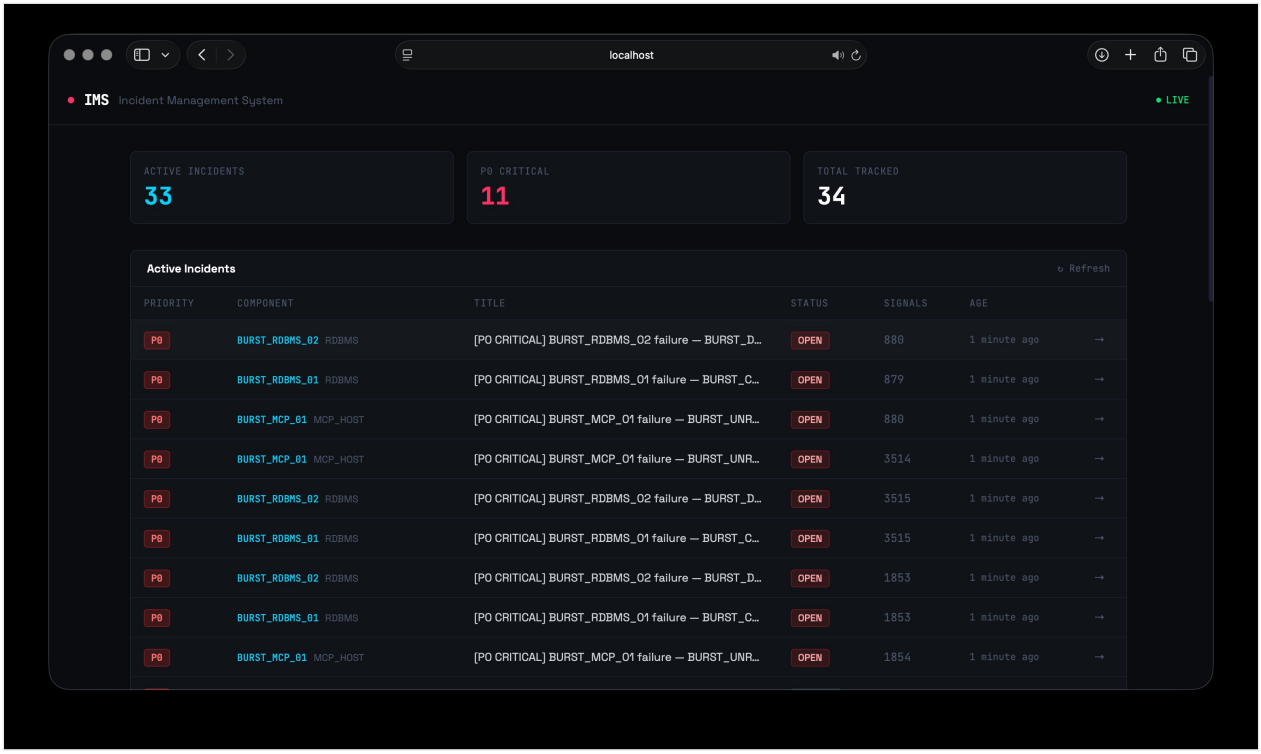

Figure 1 — Active Incidents dashboard. Stats bar shows 33 active incidents, 11 P0-Critical. The live feed is sorted P0-first; each row carries priority badge, component ID, truncated title, status, signal count, and age.

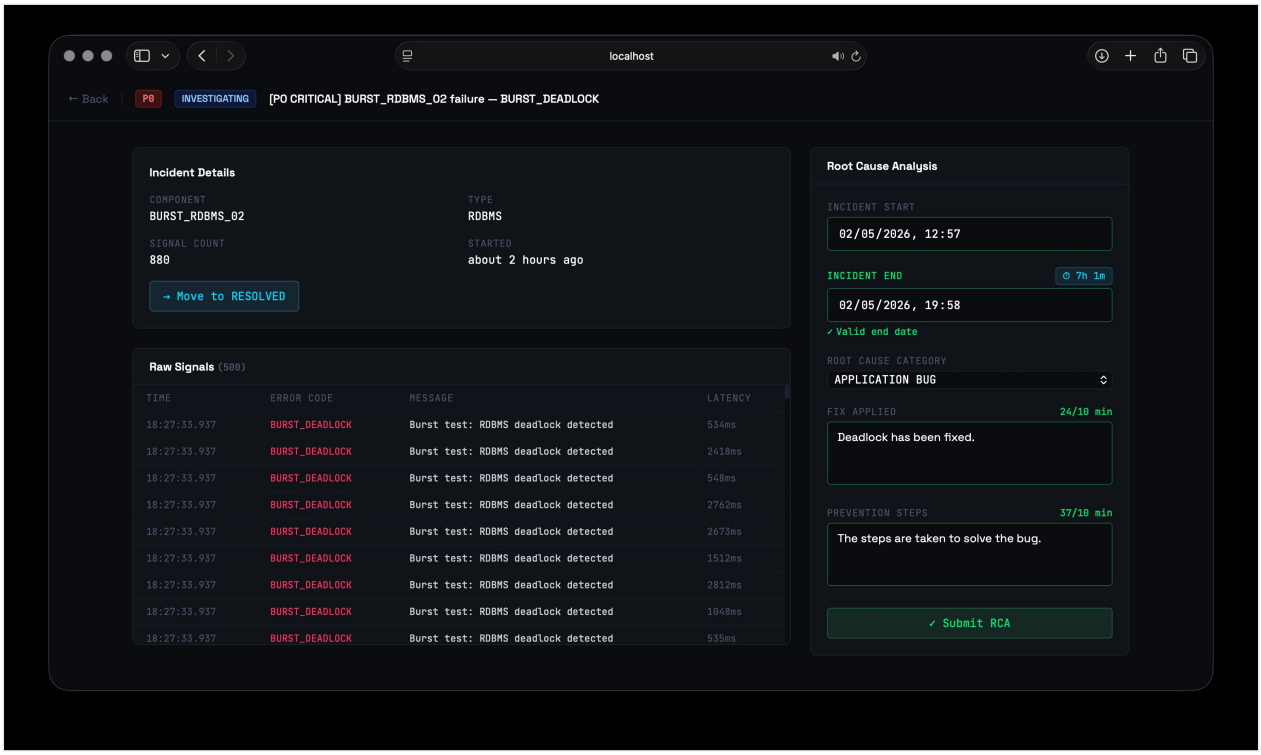

Figure 2 — Incident detail & RCA form. Left panel shows component metadata, signal count, and raw signals table (500 entries). Right panel is the RCA form with incident start/end time pickers, MTTR calculation (7 h 1 m), root-cause category dropdown, fix-applied and prevention-steps fields with live char counters, and the Submit RCA button that gates the RESOLVED → CLOSED transition.

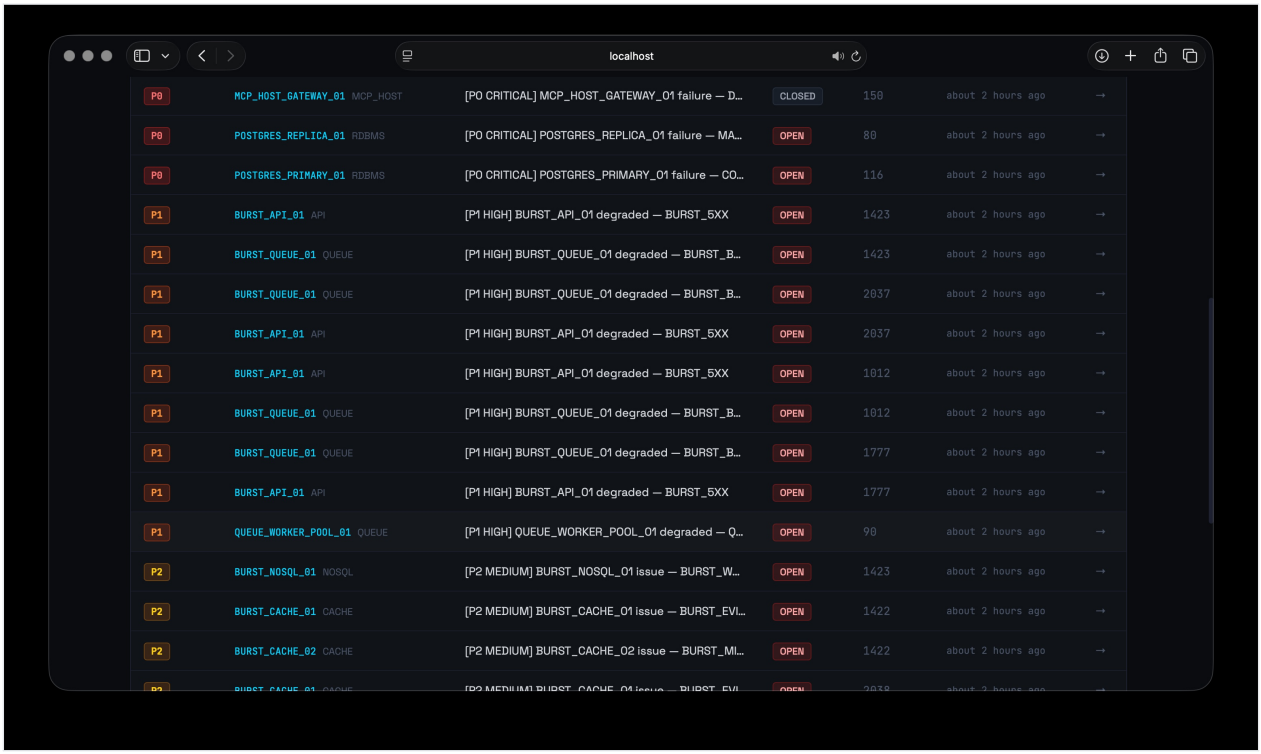

Figure 3 — Mixed-priority incident list from the cascading outage scenario. Rows span P0 (RDBMS, MCP_HOST), P1 (API, QUEUE), and P2 (NOSQL, CACHE) — demonstrating the Strategy Pattern routing different component types to correct alert priorities. One incident is already CLOSED with a submitted RCA.

GitHub	github.com/joery0x3b800001/Mission-Critical-IMS
Submitted by	Shaun Joe Roy
Submitted to	karan.sinha@zeotap.com — Zeotap Talent Team